COMPUTER MUSIC
LANGUAGES AND SYSTEMS

Jacob SuperCollier

Report

Angelica Cagnetta 10941744

Giuliano Di Lorenzo 10986806

Nicola Mugnaini 10684409

Ernest Ouali 10984484

POLITECNICO
MILANO 1863

Abstract

The purpose of this report is to document the structure and the creation process of `Jacob SuperCollier`, a computer music system for dance-based interactive sound synthesis for MIDI instruments.
The system includes different environments (`SuperCollider`, `JUCE` and `Processing`) and external devices (`Arduino` and various sensors) in order to interface MIDI instruments with the computer music system, to define the sound synthesis, to apply two sound effects and to make the sensors change the overall sound behavior. The goal is to allow a dance performance to change, in real-time, the sound design for a MIDI keyboard.

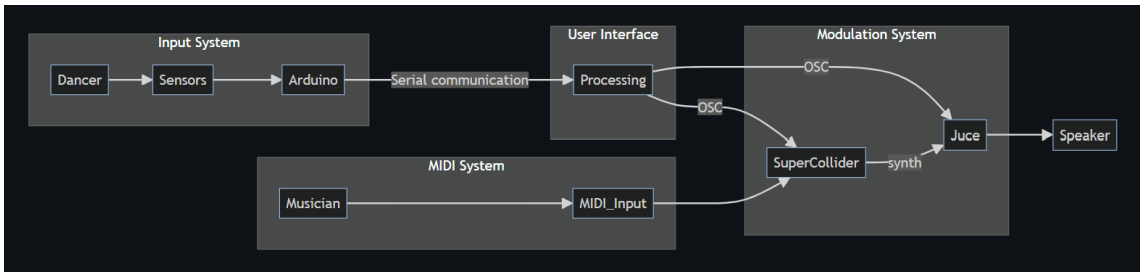

Figure 1: Block diagram of the sound chain

1 SuperCollider

`SuperCollider` is an environment for sound synthesis and algorithmic music composition, and it is based upon a real-time sound-based object-oriented programming language. It provides a framework for acoustic research, algorithmic music, interactive programming and live coding.

In this system, `SuperCollider` has been used to interface the MIDI keyboard, to define the sound synthesis and, for demonstration purposes, to schedule musical events in time.

1.1 MIDI interface

`SuperCollider` natively provides objects to manage MIDI communications, both in input and output, and to assign different functionalities to the incoming MIDI messages.

The MIDI instruments recognition and connection are respectively managed by the classes `MIDIClient` and `MIDIIn`.

`MIDIClient` allows the user to access all I/O MIDI ports, by connecting to the operating system MIDI layer. It is necessary to initialise it through the method `MIDIClient.init`.

`MIDIn`, on the other hand, is a low-level class specific for receiving MIDI messages and allows to connect all MIDI devices through the method `MIDIIn.connectAll`, so that all incoming MIDI messages can be received.

Note that `MIDIIn.connectAll` implicitly calls `MIDIClient.init`, so the first instruction may suffice for the purpose in mind.

Finally, the `MIDIFunc` class, which allows to describe functions as response to incoming MIDI messages, has the method `MIDIFunc.trace` that prints (and dumps) all incoming messages. It is very useful for debugging and experimentation purposes.

1.2 Sound synthesis

The sound synthesis has been defined by using two synthesizers and an internal audio bus, in order to implement the following sound chain:

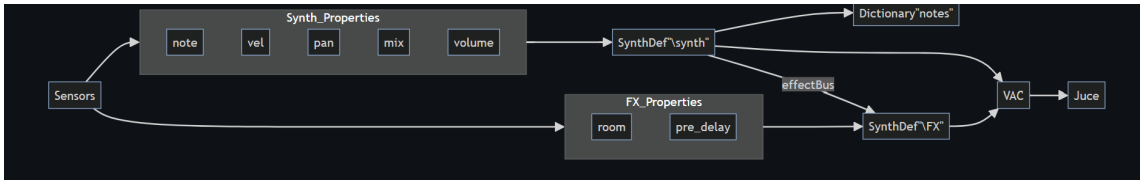

Figure 2: SuperCollider's sound chain block diagram

The `SynthDef`, called `synth`, defines the `UGens` (sound generative units) that create the sound. A `Saw` waveform generator determines the basis of the sound, with frequency and amplitude given by MIDI note number (converted in Hz) and velocity respectively. The amplitude response to the MIDI key press is given by an `Env.adsr` envelope. A percussive transient is described by a `PinkNoise` with a fast `Env.perc` envelope. Finally, the signal becomes stereo thanks to a `Pan2` object and a percentage `dry` is sent to the output channels, while the remaining percentage is directed to an internal audio bus.

Note that `Env.perc` in the first synth automatically decays, while `Env.adsr` waits at the sustain level until the gate parameter changes, and this peculiarity is used in the MIDI functionalities.

The `SynthDef`, called `FX`, defines the sound processing chain applied to the parallel internal audio bus `effectBus`. The idea is to resemble a basic room response by applying a pre-delay through `DelayN` (maximum 5 seconds allowed for storage purposes), a bandpass through a `HPF-LPF` combination (frequency passband of $200 - 5000$ Hz), and a stereo reverb through `FreeVerb2` (with `mix = 1.0`).

The two signal paths defined above need two audio busses. The dry signal goes directly to the hardware output stereo bus (indexes 0 and 1). The wet signal goes to a parallel stereo `Bus.audio` object called `effectBus`. The parallel bus connection occurs with a `Synth` instance of `FX`, called `effectSynth`, by giving `effectBus` as input bus and `[0,1]` as output bus.

1.3 MIDI functions

`SuperCollider` offers an intuitive way to define response functionalities to specific incoming MIDI messages through the class `MIDIdef`. It is of interest to use `noteOn` and `noteOff` messages only. `noteOn` suggests that a note should be played, with specific frequency and intensity, and that should be stopped when a `noteOff` occurs.

In order to do that, a `Dictionary` object called `notes` is defined to keep track of all notes that are playing in a determined instant of time. A dictionary is particularly useful for this purpose thanks to the possibility of linking a name to each of its entries.

The `MIDIdef.noteOn` method creates an instance of `synth` with ID name `"Synth"++num` (i.e. `"Synth60"` for a C3 note) and adds it to the dictionary through the method `put`. The frequency `freq` is given by the conversion of MIDI note numbers into Hz (through the method `midicps`), while the intensity `vel` is given by the linear-exponential mapping of MIDI velocities (in the range 0-127) into amplitude values (in the range 0-1) through the method `linexp`.

Note that any incoming `noteOn` message with `vel=0` is avoided, as conventionally considered as a `noteOff`.

The `MIDIdef.noteOff` method stops the execution of a specific note by retrieving the corresponding synth instance in `notes` (through the method `at`), by removing it from the dictionary (through the method `removeAt`) and by setting the parameter `gate=0` (through the method `set`). The synthesizer `synth`, in fact, has an ADSR envelope in amplitude, which goes into release when `gate=1` is changed into `gate=0`. The parameter `doneAction=2` imposes the synth to be freed after the release.

1.4 Backing track

For demonstration purposes, a backing track has been composed using `SuperCollider`. Different `SynthDefs` define different sounds (like drum kick, snare, bass, pad etc.). A more precise and creative approach has been used to compose a 1-minutes long song (cover of "End of Beginning" by Djo), considering that no kind of sound effect is applied to it.

In order to schedule different musical events in time, the object `Routine` has been widely used. It allows to define a function with the possibilities of pausing and resuming it. In addition to that, it is possible to use the `yield` method to separate different musical events with specific time intervals. The method `do` has been used to execute cycles of musical instructions.

The composition of all musical instruments patterns is given by different routines (through the method `r`), defining different `Routine` objects. All of them are played by using the method `play`.

`SuperCollider` offers a way to define a musical time attribute (such as the BPM) through the `TempoClock` object. It defines an internal clock in beats per second, so it is possible to obtain a musical time structure through `TempoClock(bpm/60)`. In order to schedule every musical event in "grid", the parameter `quant` specifies a quantized time instant, accordingly to the time structure, and it is used in the `play` method.

2 JUCE

JUCE is an open-source cross-platform C++ application framework, used to develop desktop and mobile applications. In particular, it offers presets and libraries for audio plugin development, available in the Projucer application. In this way, the audio plugin is written as general as possible, without any worries about the operating system it will run on.

For this system, the audio plugin has been developed as a standalone application, through the "basic plugin" preset with the addition of `juce_dsp` and `juce_osc` modules. An audio plugin developed with JUCE presents two main components, `AudioProcessor` for the plugin algorithm definition and `AudioProcessorEditor` for the GUI definition, both of them in form of one `.cpp` file and one `.h` file.

2.1 APVTS

In order to define all parameters of the plugin, an `AudioProcessorValueTreeState` has been defined in the processor as `apvts`. It allows to easily setup the communication between editor and processor and to contain all plugin's parameters. In particular, the `apvts` uses the `createParameter` method, to be defined for each plugin, in order to return a `ParameterLayout` object. Inside of this method, a vector `parameters`, of unique pointers to `RangedAudioParameter` objects, is defined by pushing all the plugin parameters in it, in the form of unique pointers to `AudioParameterFloat` objects. An `AudioParameterFloat` describes each parameter by means of ID, name, minimum value, maximum value and default value.

Regarding the GUI, once all `Slider` objects are defined, some slider's attachments are necessary in order to link the sliders to the plugin parameters. The linking is done by defining different unique pointers to `SliderAttachment` objects and specifying the parameter ID and the slider name that need to be linked.

2.2 DSP module

In order to define the plugin algorithm, the `juce_dsp` module has been included. In particular, it offers different already-made effects objects, including `Chorus` and `Phaser`.

In order to use them, it is necessary to declare them (and specify the sample type, like `juce::dsp::Chorus<float> chorus`) and include their constructor (such as `chorus()`) inside the `AudioProcessor` constructor.

Once the plugin is running, the effect objects need to be prepared in the `prepareToPlay` method, and their method `prepare` allows to do that. This method requires the sample rate, the buffer size and the number of output channels in the form of `ProcessSpec` object.

The algorithm description occurs inside the `processBlock` method, where the effects need to be set. In particular, `Chorus` and `Phaser` offer different methods (such as `setRate`) to set their internal parameters by passing the desired values. In the end, each effect object processes the input samples through their method `process`, which needs an input buffer in terms of `AudioBlock` object.

3 Arduino

The computer music system communicates with the outer world, aside from MIDI devices, with the electronic board `Arduino Uno WiFi Rev2.` It offers the possibility of interfacing various external devices, such as sensors, through a breadboard and various digital and analog pins.

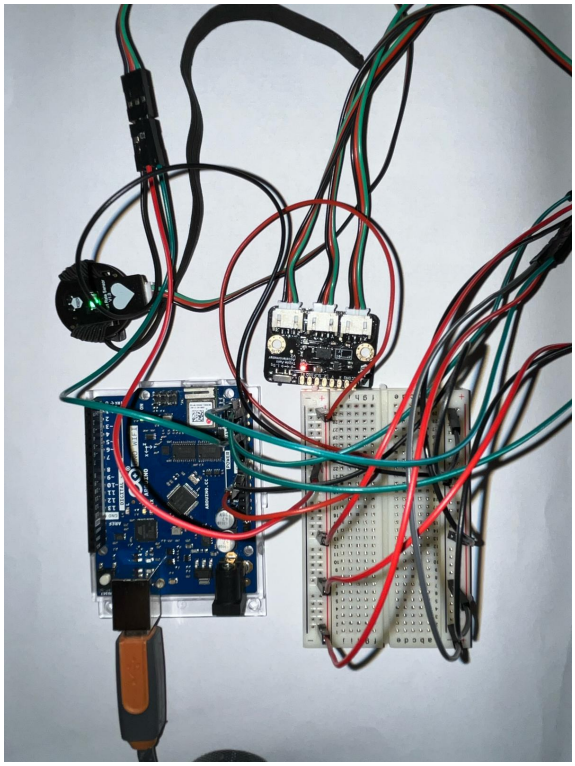

Figure 3: Arduino setup

3.1 Sensors

In this system, a triple axis accelerometer and a heart rate monitor (by `DFRobot`) are connected to a dancer and their signals are captured by the `Arduino` board. The triple axis accelerometer returns three accelerations expressed in m/s^2, thanks to a signal processing script converting measured voltages into the usual unit for acceleration. The heart rate monitor can work in analog or digital mode: in analog mode it returns a duty cycle value in ms.

3.2 Libraries and code

In order for the `Arduino` code to efficiently communicate with `Processing`, we implemented a Serial communication. Processing will have to listen at the appropriate COM port of the computer in order to retrieve the 4 values of interest: accelerations and heart rate. The user should check that the detected COM port from the `Arduino` software is the same one indicated in the `Processing` script (in our case COM6).
We downloaded a library developed by the user *jeroendoggen* found on `GitHub` that is designed to use the triple-axis accelerometer. This helped us to better understand how to interact with the sensor in Arduino. The `Arduino` script is quite minimalistic as we are only looking to transmit information.

4 Processing

4.1 GUI

Thanks to the library `G4P`, developed by Peter Lager, we were able to easily and intuitively create an appropriate user interface that would respond to all of our needs. The `gui.pde` file is automatically generated by the library. We only specified the handlers behavior: the user can select which parameter is influenced by a specific signal coming from `Arduino`, and the script is modifying the relevant parameters, allowing the dancer to keep performing while the signal processing is modified in real-time. More specifically, as the X-position and Y-position share the exact same parameters modifiers, the handlers functions we developed ensure that the said positions cannot modify the same parameter at the same time. Look at `Panning_X_position_event()` to see how this conflict is typically avoided.

The GUI consists of a main panel where the user selects which parameter should be modified by a given signal received from the sensors. In the middle of the interface, a canvas is representing the dancer's approximate position in space. This helps to have a rough estimation of the parameters being modified as the position of the dancer is the key modifier. It also helped us to see if our sensor was correctly set. We included `if` and `else if` statements to restrict the virtual dancer in the given room.

Note that each time we run the `Processing` script, the dancer's position is set to the middle of the room.

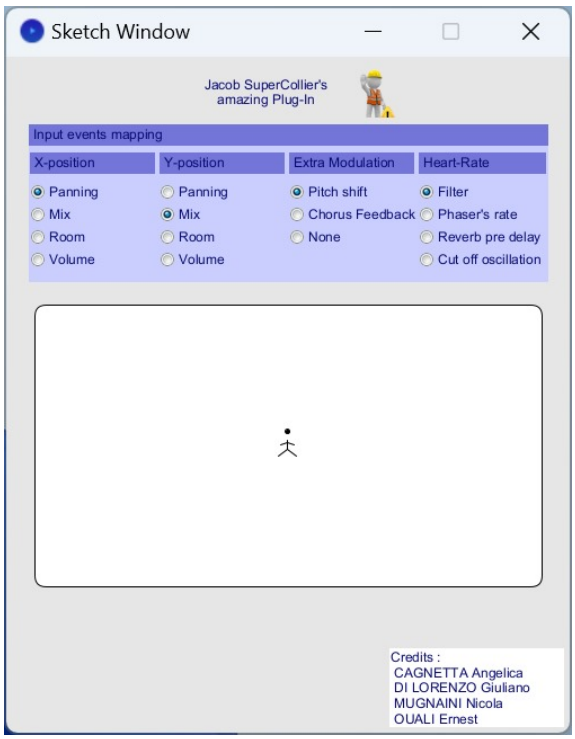

Figure 4: GUI design

4.2 Serial Communication

As mentioned before, since `Arduino` is sending Serial messages, we had to configure a `Serial` receiver in the `Processing` sketch. This was easily achieved thanks to the built-in library `processing.serial`. The script is methodically extracting the data received on the `Serial` port and storing each relevant information into a specific variable. See the `harvestSerial()` function in the `Arduino` script for more details.

4.3 OSC Messages

In order for `Processing` to inform `SuperCollider` and `JUCE` about the parameters modified by the dancer, we implemented an `OSC` message sender thanks to the libraries `oscP5`, and `netP5`. All `OSC` messages are transmitted on the local machine. We constructed several `OSC` messages in the script but only the last one, called `"/Effects_Values"`, is relevant for both software.

More precisely the said message contains different arguments, which are ordered as :

- `panning`,
- `mix`,
- `room`,
- `volume`,
- `filter_freq`,
- `pitch_shift`,
- `phaser_rate`,
- `rev_pre_delay`,
- `chorus_fb`,
- `oscillation_range`

See the `controlEvent()` function in the `Processing` script for more details.

4.4 Parameter Computation

In order to respect the way that `SuperCollider` and `JUCE` are interpreting the modulation effects values, we implemented functions which translate the received signals into bounded values. To do so, we simply mapped the received signals into a linear relationship of the shape $ax + b$ with extreme values.

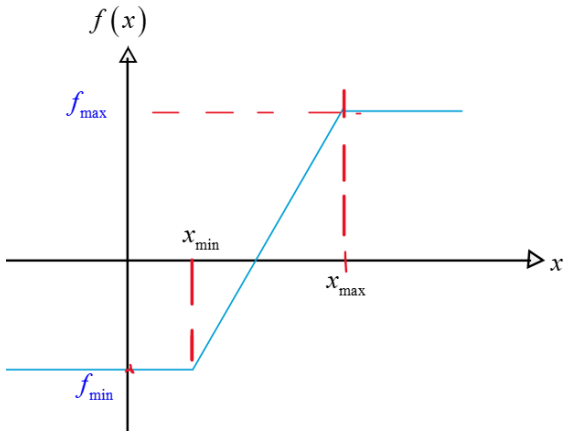

Figure 5: Linear relationship between the position x of the dancer and the parameter's value $f(x)$

The general way of defining the linear relationship in the code respected the following function definition. Here, we arbitrarily take the x position as the studied modifier :

$$\begin{aligned} &\forall x \in [x_{\min}; x_{\max}], f(x) = ax + b \\ &\forall x < x_{\min}, \qquad f(x) = f_{\min} \\ &\forall x > x_{\max}, \qquad f(x) = f_{\max} \end{aligned}$$

Based on those specifications, we easily find that:

$$b = \frac{f_{\min} x_{\max} - f_{\max} x_{\min}}{x_{\max} - x_{\min}}$$
$$a = \frac{f_{\max} - f_{\min}}{x_{\max} - x_{\min}}$$

We find an analog expression of the linear relationship for the y position. Thus, we implemented 4 functions, that you can have a look at, to better understand how the parameter computation is achieved :

- `change_X_pos_mapping()`
- `change_Y_pos_mapping()`
- `change_other_mod()`
- `change_HR_mapping()`

Some parameters computation does not go through this linear mapping. For example, the `pitch_shift` parameter follows a non-linear relationship. Depending on the position of the dancer in the room, `pitch_shift` takes the values as depicted below:

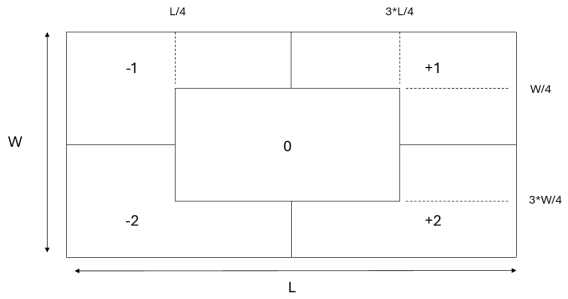

Figure 6: Semitone shift depending on the location of the dancer in the room

5 Credits

This computer system is the result of the project work for "Computer Music - Languages and Systems" exam at Politecnico di Milano, made by "Jacob SuperCollier" team.

"Jacob SuperCollier" is:

- Cagnetta Angelica
- Di Lorenzo Giuliano
- Mugnaini Nicola
- Ouali Ernest